# Fiskario Mobile - Enhanced Backend Integration

## Overview

This document describes the enhanced backend integration implemented for the Fiskario mobile application, providing comprehensive invoice and customer operations with offline support and tax calculation integration.

## Features Implemented

### 1. Enhanced API Integration

#### New Mobile-Optimized Endpoints

- **Tax Calculation**: `POST /invoicing/mobile/calculate` - Real-time tax calculations with Polish VAT rules
- **Invoice Preview**: `POST /invoicing/mobile/preview` - Preview invoices before creation
- **Invoice Validation**: `POST /invoicing/mobile/validate` - Validate invoice data against business rules
- **Invoice Templates**: `GET /invoicing/mobile/templates/:companyId` - Company-specific invoice templates

#### Mobile Sync System

- **Full Sync**: `POST /mobile-sync/full-sync` - Complete data synchronization
- **Incremental Sync**: `POST /mobile-sync/incremental-sync` - Sync only changed data
- **Conflict Resolution**: `POST /mobile-sync/resolve-conflict` - Handle data conflicts
- **Pending Changes**: `GET /mobile-sync/pending-changes/:companyId` - Get unsynchronized changes

### 2. Mobile-Optimized Invoice Management

#### Enhanced Invoice Screen Features

- **Real-time Data**: Invoices loaded from backend API instead of mock data
- **Pull-to-Refresh**: Sync with backend on user gesture
- **Error Handling**: Comprehensive error states with user-friendly messages
- **Loading States**: Proper loading indicators for all operations
- **Offline Queue**: Visual indicators for invoices pending sync

#### Invoice Creation Flow

1. **Customer Selection**: Choose from existing customers or create new ones
2. **Item Management**: Add items with automatic tax calculations
3. **Tax Preview**: Real-time tax calculation and breakdown
4. **Validation**: Comprehensive validation before submission

5. **Offline Support**: Queue for sync when offline

## 3. Customer Management Integration

### Customer Model

```
interface Customer {
  id: string;
  name: string;
  nip: string; // Polish tax identification number
  address: string;
  city: string;
  postalCode: string;
  country: string;
  email: string;
  phone: string;
  isActive: boolean;
  paymentTerms: number; // days
  notes: string;
  serverId?: string;
  lastSync?: number;
}
```

### Customer Service Features

- **CRUD Operations**: Create, read, update, delete customers
- **NIP Validation**: Polish NIP number validation algorithm
- **Search**: Search by name or NIP number
- **Sync**: Synchronize with backend customer database

## 4. Tax Calculation System

### Supported Features

- **Multiple VAT Rates**: 0%, 5%, 8%, 23% (Polish standard rates)
- **GTU Codes**: Polish tax codes for specific product categories
- **Tax Breakdown**: Detailed breakdown by VAT rate
- **Applied Rules**: Track which tax rules were applied

### Tax Calculation Request

```
interface TaxCalculationRequest {
  companyId: string;
  items: TaxCalculationItem[];
  invoiceType?: 'VAT' | 'proforma' | 'correction';
  includeAllTaxes?: boolean;
```

```
  additionalData?: Record<string, any>;
}
```

### 5. Offline Support

**Offline Invoice Service**

- **Queue Management**: Queue invoices for sync when offline
- **Conflict Resolution**: Handle server/client conflicts
- **Auto-sync**: Automatically sync when connection restored
- **Data Cleanup**: Remove old offline data (30+ days)

**Sync Strategies**

- **Client Wins**: Local changes override server data
- **Server Wins**: Server data overrides local changes
- **Manual Merge**: User resolves conflicts manually

### 6. Error Handling and User Feedback

**Mobile Error Handler**

- **Error Classification**: Network, validation, auth, sync errors
- **User-Friendly Messages**: Localized error messages in Polish
- **Retry Logic**: Automatic retry for recoverable errors
- **Error Logging**: Comprehensive error logging for debugging

**Error Types**

- **NETWORK_ERROR**: Connection issues
- **VALIDATION_ERROR**: Invalid data
- **AUTH_ERROR**: Authentication problems
- **SYNC_ERROR**: Synchronization failures
- **CALCULATION_ERROR**: Tax calculation errors

## Usage Examples

### Creating an Invoice with Tax Calculation

```
import { taxCalculationService } from '../services/taxCalculationService';

const invoiceRequest = {
  companyId: 'company-123',
  items: [
    {
      description: 'Consulting Services',
      quantity: 40,
```

```
      unitPrice: 150,
      vatRate: 23,
      gtu: 'GTU_12',
    }
  ],
  invoiceType: 'VAT' as const,
};

const calculation = await taxCalculationService.calculateTaxes(invoiceRequest);
console.log(calculation);
// {
//   totalNet: 6000,
//   totalVat: 1380,
//   totalGross: 7380,
//   vatBreakdown: [...],
//   appliedRules: [...],
//   success: true
// }
```

### Managing Customers

```
import CustomerService from '../services/customerService';
import { database } from '../database/database';

const customerService = CustomerService.getInstance(database);

// Create customer
const customerId = await customerService.createCustomer({
  name: 'ABC Company Sp. z o.o.',
  nip: '1234567890',
  address: 'Business Street 1',
  city: 'Warsaw',
  postalCode: '00-001',
  country: 'Poland',
  email: 'contact@abc.com',
  phone: '+48 123 456 789',
  isActive: true,
  paymentTerms: 14,
  notes: 'Valued customer',
});

// Search customers
const customers = await customerService.searchCustomers('company-123', 'ABC');
```

### Handling Offline Scenarios

```
import OfflineInvoiceService from '../services/offlineInvoiceService';
import { database } from '../database/database';
```

```
const offlineService = new OfflineInvoiceService(database);

// Save for offline
await offlineService.saveInvoiceOffline(invoiceData);

// Queue for sync
offlineService.queueForSync(invoiceData);

// Check if online and sync
if (await offlineService.isOnline()) {
  const result = await offlineService.syncWithServer(syncFunction);
  console.log(`Synced: ${result.synced}, Failed: ${result.failed}`);
}
```

## Testing

### Running Integration Tests

```
npm test integration.test.ts
```

### Test Coverage

- ☑ Error Handler Integration
- ☑ Tax Calculation Service
- ☑ Offline Invoice Service
- ☑ Customer Service
- ☑ API Integration Points
- ☑ Error Recovery
- ☑ Data Consistency

## Configuration

### Environment Variables

```
API_BASE_URL=http://localhost:3000/api
SYNC_INTERVAL=30000
OFFLINE_QUEUE_SIZE=100
TAX_CALCULATION_TIMEOUT=5000
```

### Database Schema Updates

The following models have been added/updated:

- **Customer**: New model for customer management

- **Invoice**: Enhanced with server sync fields
- **InvoiceItem**: Enhanced with GTU codes and categories

# Migration Guide

## From Mock Data to Real API

1. **Update Imports**: Replace mock data imports with API hooks
2. **Add Error Handling**: Wrap API calls with error handlers
3. **Implement Loading States**: Add loading indicators for all API operations
4. **Add Offline Support**: Queue operations when offline
5. **Update UI**: Handle empty states and error states

## Breaking Changes

- Invoice screen now requires company context
- Customer selection is mandatory for invoice creation
- Tax calculations are performed server-side
- All operations support offline mode

# Troubleshooting

## Common Issues

1. **Network Errors**
   - Check internet connection
   - Verify API_BASE_URL configuration
   - Check for firewall/proxy issues
2. **Sync Conflicts**
   - Use conflict resolution API
   - Check server vs client data timestamps
   - Manual merge for complex conflicts
3. **Tax Calculation Errors**
   - Verify VAT rates are valid (0, 5, 8, 23)
   - Check GTU codes format
   - Validate item quantities and prices
4. **Offline Data Issues**
   - Clear old offline data periodically
   - Check available storage space
   - Verify database integrity

## Debug Mode

Enable debug logging:

```
mobileErrorHandler.handleError(error, 'context', {
  showAlert: true,
  logError: true,
});
```

## Performance Considerations

- **Batch Operations**: Group multiple API calls
- **Caching**: Cache frequently accessed data
- **Lazy Loading**: Load data on demand
- **Background Sync**: Sync data in background
- **Storage Limits**: Monitor offline queue size

## Security

- **Data Encryption**: Sensitive data encrypted in storage
- **API Authentication**: JWT tokens for all API calls
- **Input Validation**: Server-side validation for all inputs
- **Audit Logging**: Track all data modifications

## Future Enhancements

- ☐ Push notifications for invoice status changes
- ☐ Advanced reporting and analytics
- ☐ Multi-language support
- ☐ Advanced search and filtering
- ☐ Integration with external systems
- ☐ Advanced offline conflict resolution UI

---

For questions or support, please contact the development team.